

EXERCISE 1: Configuring a Basic Spring Application

Scenario:

Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.

Steps:

1. **Set Up a Spring Project:**
 - Create a Maven project named **LibraryManagement**.
 - Add Spring Core dependencies in the **pom.xml** file.
2. **Configure the Application Context:**
 - Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
 - Define beans for **BookService** and **BookRepository** in the XML file.
3. **Define Service and Repository Classes:**
 - Create a package **com.library.service** and add a class **BookService**.
 - Create a package **com.library.repository** and add a class **BookRepository**.
4. **Run the Application:**
 - Create a main class to load the Spring context and test the configuration.

Solution Code:

BookService.java

```
package com.library.service;
import com.library.repository.BookRepository;
public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void getBooks() {
        System.out.println("Book Service Called");
        bookRepository.displayBooks();
    }
}
```

```
h.xml (LibraryManagemetn)  BookService.java x  applicationContext.xml  Main.java
1  package com.library.service;
2  import com.library.repository.BookRepository;
3  public class BookService { 4 usages
4
5      private BookRepository bookRepository; 2 usages
6
7      public void setBookRepository(BookRepository bookRepository) { 2 usag
8          this.bookRepository = bookRepository;
9      }
10
11     public void getBooks() { 1 usage
12         System.out.println("Book Service Called");
13         bookRepository.displayBooks();
14     }
15 }
```

BookRepository.java

```
package com.library.repository;
```

```
public class BookRepository {
```

```
    public void displayBooks() {
        System.out.println("Fetching books from repository...");
    }
}
```

```
m.xml (LibraryManagemetn)  BookService.java  applicationContext.xml  Main.java
1  package com.library.repository;
2
3  public class BookRepository { 4 usages
4
5      public void displayBooks() { 1 usage
6          System.out.println("Fetching books from repository...");
7      }
8  }
```

Main.java

```
import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {

    public static void main(String[] args) {
        ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
        BookService bookService = context.getBean("bookService", BookService.class);
        bookService.getBooks();
    }
}
```



applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>

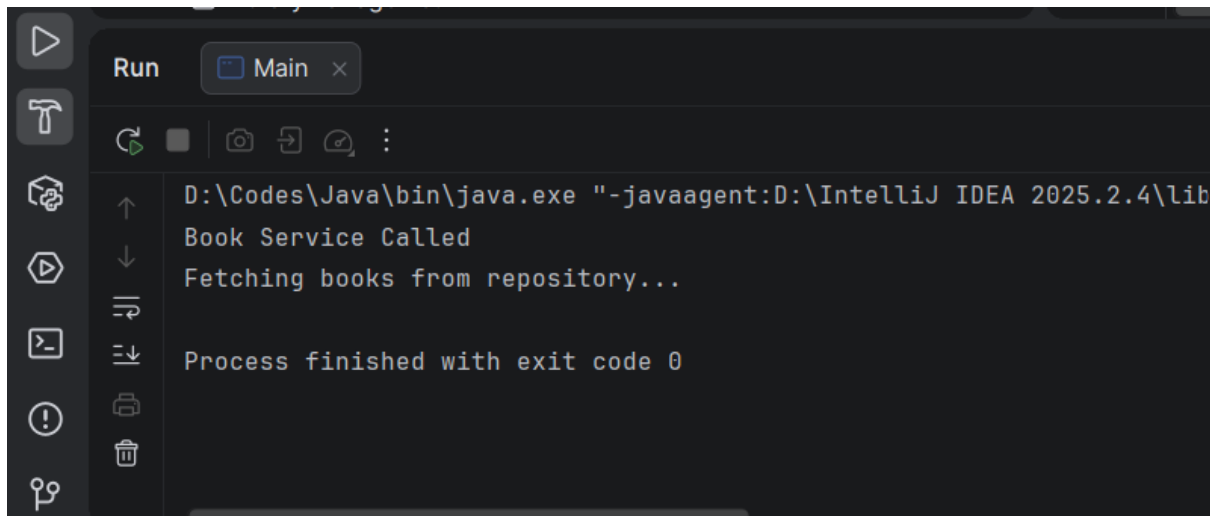
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService">

        <property name="bookRepository"
                  ref="bookRepository"/></bean></beans>
```

OUTPUT:



```
Run Main x
D:\Codes\Java\bin\java.exe "-javaagent:D:\IntelliJ IDEA 2025.2.4\lib
Book Service Called
Fetching books from repository...
Process finished with exit code 0
```

EXERCISE 2 : Implementing Dependency Injection

Scenario:

In the library management application, you need to manage the dependencies between the `BookService` and `BookRepository` classes using Spring's IoC and DI.

Steps:

1. **Modify the XML Configuration:**
 - Update `applicationContext.xml` to wire `BookRepository` into `BookService`.
2. **Update the `BookService` Class:**
 - Ensure that `BookService` class has a setter method for `BookRepository`.
3. **Test the Configuration:**
 - Run the `LibraryManagementApplication` main class to verify the dependency injection.

Solution Code:

`BookService.java`

```
package com.library.service;
import com.library.repository.BookRepository;
public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }
}
```

```

    }

    public void getBooks() {
        System.out.println("Book Service Called");
        bookRepository.displayBooks();
    }
}

```

```

1  package com.library.service;
2  import com.library.repository.BookRepository;
3  public class BookService { 4 usages
4
5      private BookRepository bookRepository; 2 usages
6
7      public void setBookRepository(BookRepository bookRepository) { 2 usag
8          this.bookRepository = bookRepository;
9      }
10
11     public void getBooks() { 1 usage
12         System.out.println("Book Service Called");
13         bookRepository.displayBooks();
14     }
15 }

```

BookRepository.java

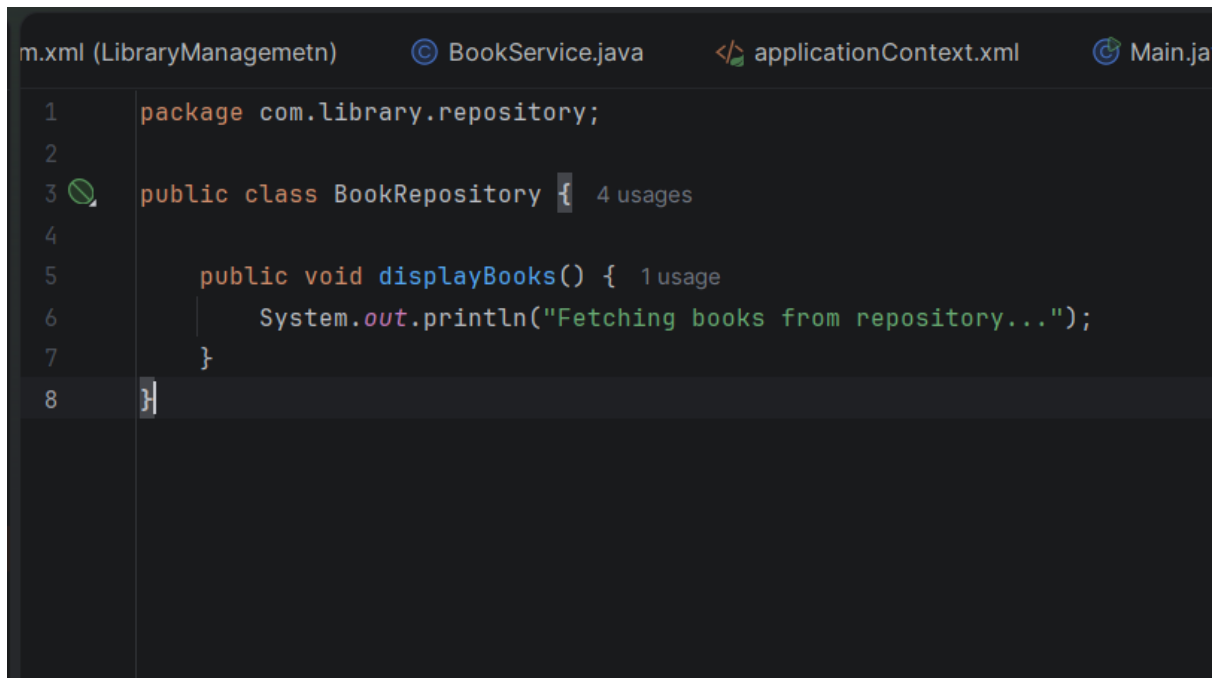
```

package com.library.repository;

public class BookRepository {

    public void displayBooks() {
        System.out.println("Fetching books from repository...");
    }
}

```



```
m.xml (LibraryManagemetn)  BookService.java  applicationContext.xml  Main.java
1  package com.library.repository;
2
3  public class BookRepository { 4 usages
4
5      public void displayBooks() { 1 usage
6          System.out.println("Fetching books from repository...");
7      }
8  }
```

Main.java

```
import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {

    public static void main(String[] args) {
        ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
        BookService bookService = context.getBean("bookService", BookService.class);
        bookService.getBooks();
    }
}
```



```
m.xml (LibraryManagemetn)  BookService.java  applicationContext.xml  Main.java x  BookRepository.java
1  import com.library.service.BookService;
2  import org.springframework.context.ApplicationContext;
3  import org.springframework.context.support.ClassPathXmlApplicationContext;
4
5  public class Main {
6
7      public static void main(String[] args) {
8          ApplicationContext context = new ClassPathXmlApplicationContext( configLocation: "applicationContext.xml")
9          BookService bookService = context.getBean( s: "bookService", BookService.class);
10         bookService.getBooks();
11     }
12 }
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService">

        <property name="bookRepository"
                  ref="bookRepository"/></bean></beans>
```

OUTPUT:



```
Run Main x
D:\Codes\Java\bin\java.exe "-javaagent:D:\IntelliJ IDEA 2025.2.4\lib
Book Service Called
Fetching books from repository...
Process finished with exit code 0
```

EXERCISE 4: Creating and Configuring a Maven Project

Scenario:

You need to set up a new Maven project for the library management application and add Spring dependencies.

Steps:

1. **Create a New Maven Project:**

- Create a new Maven project named **LibraryManagement**.
- 2. **Add Spring Dependencies in pom.xml:**
 - Include dependencies for Spring Context, Spring AOP, and Spring WebMVC.
- 3. **Configure Maven Plugins:**
 - Configure the Maven Compiler Plugin for Java version 1.8 in the pom.xml file.

Solution Code:

Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.solutions.cognizant</groupId>
  <artifactId>LibraryManagemetn</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>24</maven.compiler.source>
    <maven.compiler.target>24</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.39</version>
    </dependency>

    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.39</version>
    </dependency>

    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-aop</artifactId>
      <version>5.3.39</version>
```



```
</dependency>
```

```
<dependency>
```

```
  <groupId>org.springframework</groupId>
```

```
  <artifactId>spring-webmvc</artifactId>
```

```
  <version>5.3.39</version>
```

```
</dependency>
```

```
</dependencies>
```

```
<build>
```

```
  <plugins>
```

```
    <plugin>
```

```
      <groupId>org.apache.maven.plugins</groupId>
```

```
      <artifactId>maven-compiler-plugin</artifactId>
```

```
      <version>3.11.0</version>
```

```
      <configuration>
```

```
        <source>1.8</source>
```

```
        <target>1.8</target>
```

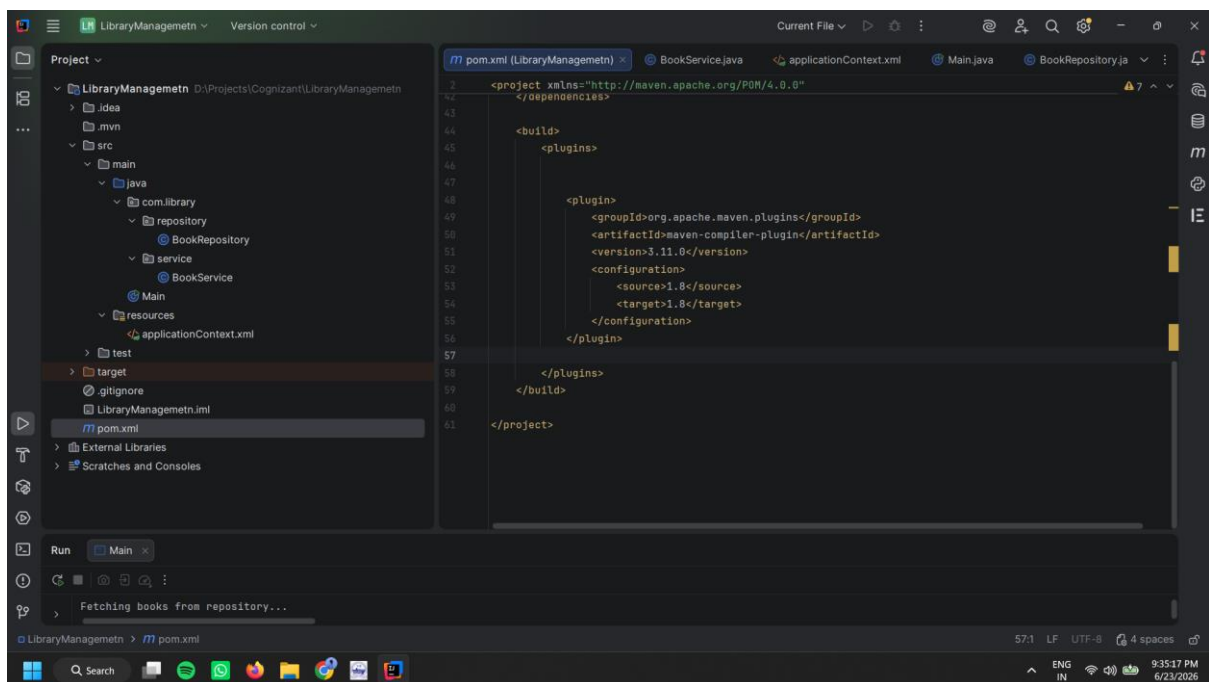
```
      </configuration>
```

```
    </plugin>
```

```
  </plugins>
```

```
</build>
```

```
</project>
```



Output:

```
Terminal Local x + v
Downloaded from central: https://repo.maven.apache.org/maven2/org/slf4j/slf4j-api/1.7.36/slf4j-api-1.7.36.jar (41 kB at 48 kB/s)
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ LibraryManagement ---
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-incremental/1.1/maven-shared-incremental-1.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-incremental/1.1/maven-shared-incremental-1.1.jar (14 kB at 43 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-component-annotations/1.5.5/plexus-component-annotations-1.5.5.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-java/1.1.2/plexus-java-1.1.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.4/asm-9.4.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compiler-api/2.13.0/plexus-compiler-api-2.13.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/com/thoughtworks/qdox/qdox/2.0.3/qdox-2.0.3.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compiler-api/2.13.0/plexus-compiler-api-2.13.0.jar (55 kB at 421 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.5.0/plexus-utils-3.5.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/com/thoughtworks/qdox/qdox/2.0.3/qdox-2.0.3.jar (334 kB at 3.5 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compiler-manager/2.13.0/plexus-compiler-manager-2.13.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.4/asm-9.4.jar (122 kB at 1.2 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compiler-javac/2.13.0/plexus-compiler-javac-2.13.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-component-annotations/1.5.5/plexus-component-annotations-1.5.5.jar (4.2 kB at 37 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.5.0/plexus-utils-3.5.0.jar (267 kB at 1.6 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compiler-manager/2.13.0/plexus-compiler-manager-2.13.0.jar (4.7 kB at 25 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compiler-javac/2.13.0/plexus-compiler-javac-2.13.0.jar (23 kB at 61 kB/s)
[INFO] Changes detected - recompiling the module! -source
[INFO] Compiling 3 source files with javac [debug target 1.8] to target/classes
[WARNING] bootstrap class path is not set in conjunction with -source 8
not setting the bootstrap class path may lead to class files that cannot run on JDK 8
--release 8 is recommended instead of -source 8 -target 1.8 because it sets the bootstrap class path automatically
[WARNING] source value 8 is obsolete and will be removed in a future release
[WARNING] target value 8 is obsolete and will be removed in a future release
[WARNING] To suppress warnings about obsolete options, use -Xlint:-options.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.779 s
[INFO] Finished at: 2024-06-23T21:36:20+05:30
[INFO] -----
PS D:\Projects\Cognizant\LibraryManagement>
```

EXERCISE 5: Configuring the Spring IoC Container

Scenario:

The library management application requires a central configuration for beans and dependencies.

Steps:

1. **Create Spring Configuration File:**
 - Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
 - Define beans for **BookService** and **BookRepository** in the XML file.
2. **Update the BookService Class:**
 - Ensure that the **BookService** class has a setter method for **BookRepository**.
3. **Run the Application:**
 - Create a main class to load the Spring context and test the configuration.

Solution Code:

BookService.java

```
package com.library.service;
import com.library.repository.BookRepository;
public class BookService {
```

```

private BookRepository bookRepository;

public void setBookRepository(BookRepository bookRepository) {
    this.bookRepository = bookRepository;
}

public void getBooks() {
    System.out.println("Book Service Called");
    bookRepository.displayBooks();
}
}

```

```

1 package com.library.service;
2 import com.library.repository.BookRepository;
3 public class BookService { 4 usages
4
5     private BookRepository bookRepository; 2 usages
6
7     public void setBookRepository(BookRepository bookRepository) { 2 usag
8         this.bookRepository = bookRepository;
9     }
10
11     public void getBooks() { 1 usage
12         System.out.println("Book Service Called");
13         bookRepository.displayBooks();
14     }
15 }

```

BookRepository.java

```

package com.library.repository;
public class BookRepository {

    public void displayBooks() {
        System.out.println("Fetching books from repository...");
    }
}

```

```
m.xml (LibraryManagemetn)  BookService.java  applicationContext.xml  Main.java
1 package com.library.repository;
2
3 public class BookRepository { 4 usages
4
5     public void displayBooks() { 1 usage
6         System.out.println("Fetching books from repository...");
7     }
8 }
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
```

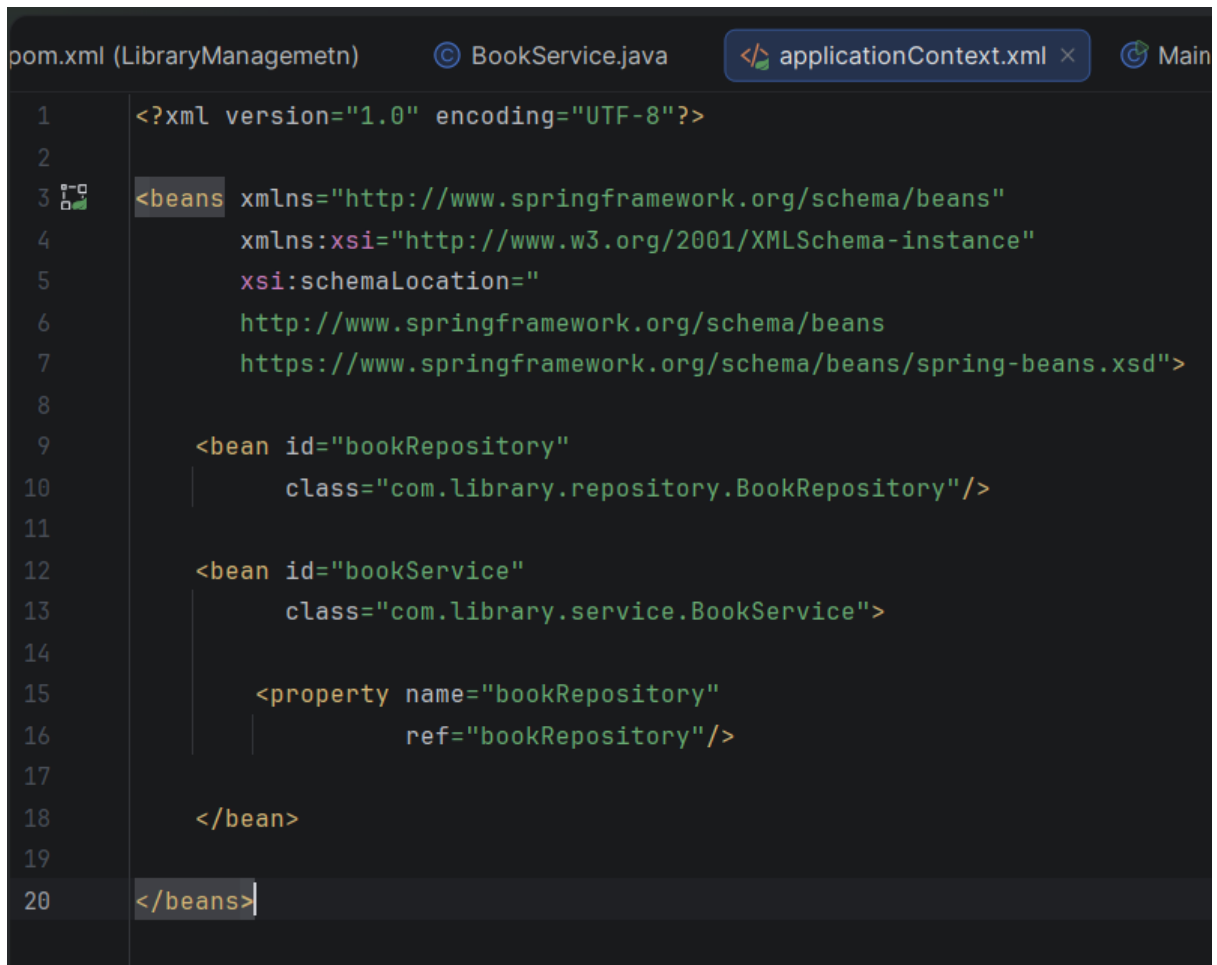
```
  <bean id="bookRepository"
    class="com.library.repository.BookRepository"/>
```

```
  <bean id="bookService"
    class="com.library.service.BookService">
```

```
    <property name="bookRepository"
      ref="bookRepository"/>
```

```
  </bean>
```

```
</beans>
```



```
1      <?xml version="1.0" encoding="UTF-8"?>
2
3      <beans xmlns="http://www.springframework.org/schema/beans"
4             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5             xsi:schemaLocation="
6                 http://www.springframework.org/schema/beans
7                 https://www.springframework.org/schema/beans/spring-beans.xsd">
8
9         <bean id="bookRepository"
10              class="com.library.repository.BookRepository"/>
11
12         <bean id="bookService"
13              class="com.library.service.BookService">
14
15             <property name="bookRepository"
16                       ref="bookRepository"/>
17
18         </bean>
19
20     </beans>
```

Main.java

```
import com.library.service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {

    public static void main(String[] args) {
        ApplicationContext context = new
        ClassPathXmlApplicationContext("applicationContext.xml");
        BookService bookService = context.getBean("bookService", BookService.class);
        bookService.getBooks();
    }
}
```

```
pom.xml (LibraryManagement)  BookService.java  applicationContext.xml  Main.java x  BookRepository.java  v  :
1  import com.library.service.BookService;
2  import org.springframework.context.ApplicationContext;
3  import org.springframework.context.support.ClassPathXmlApplicationContext;
4
5  public class Main {
6
7      public static void main(String[] args) {
8          ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
9          BookService bookService = context.getBean("bookService", BookService.class);
10         bookService.getBooks();
11     }
12 }
```

EXERCISE 7: Implementing Constructor and Setter Injection

Scenario:

The library management application requires both constructor and setter injection for better control over bean initialization.

Steps:

1. **Configure Constructor Injection:**
 - Update `applicationContext.xml` to configure constructor injection for **BookService**.
2. **Configure Setter Injection:**
 - Ensure that the **BookService** class has a setter method for **BookRepository** and configure it in `applicationContext.xml`.
3. **Test the Injection:**

Run the **LibraryManagementApplication** main class to verify both constructor and setter injection.

Solution Code:

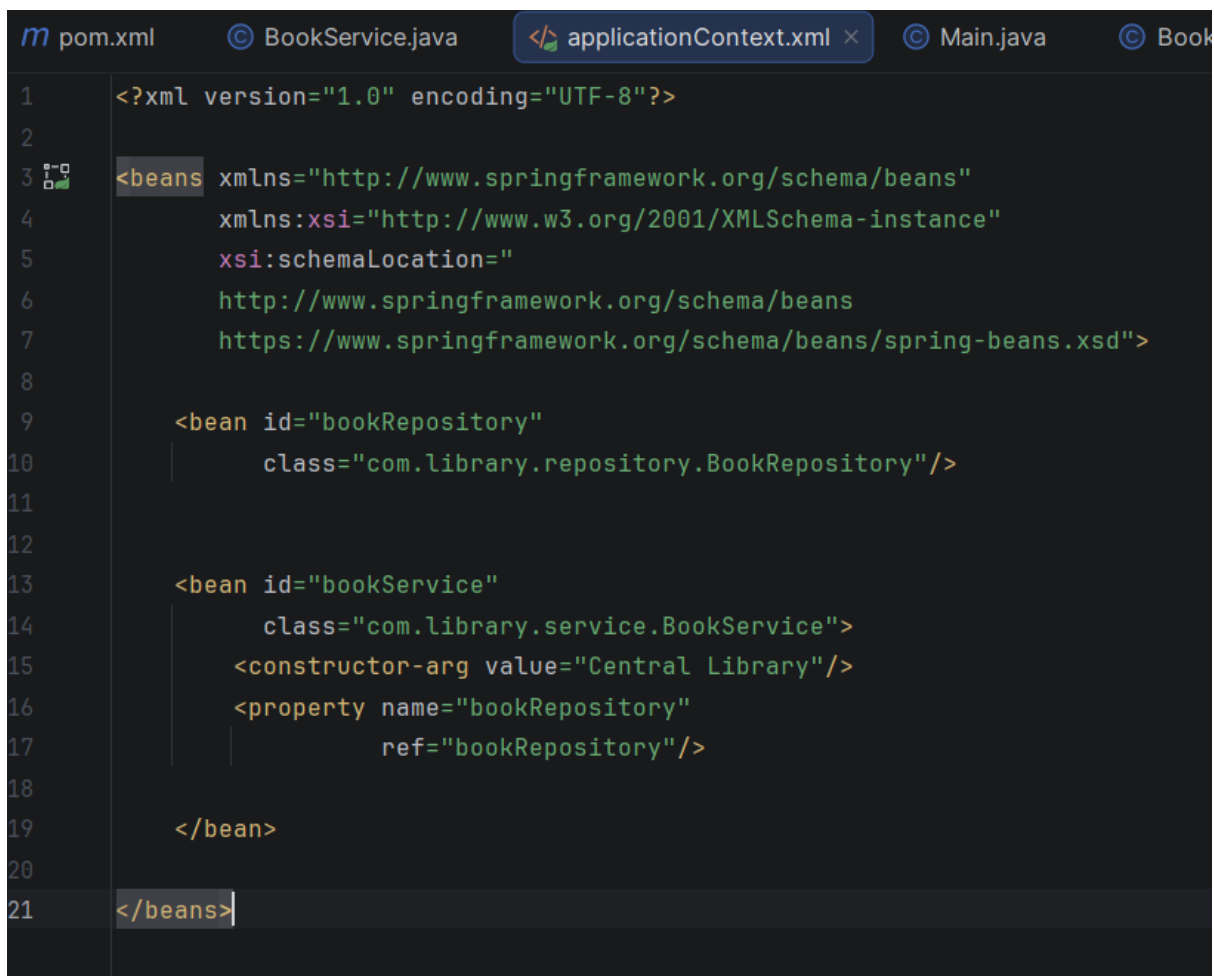
`applicationContext.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">
```

```

<bean id="bookRepository"
      class="com.library.repository.BookRepository"/>
<bean id="bookService"
      class="com.library.service.BookService">
  <constructor-arg value="Central Library"/>
  <property name="bookRepository"
            ref="bookRepository"/>
</bean>
</beans>

```



```

1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <beans xmlns="http://www.springframework.org/schema/beans"
4        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5        xsi:schemaLocation="
6          http://www.springframework.org/schema/beans
7          https://www.springframework.org/schema/beans/spring-beans.xsd">
8
9      <bean id="bookRepository"
10         class="com.library.repository.BookRepository"/>
11
12
13     <bean id="bookService"
14         class="com.library.service.BookService">
15       <constructor-arg value="Central Library"/>
16       <property name="bookRepository"
17                 ref="bookRepository"/>
18     </bean>
19
20
21 </beans>

```

BookService.java

```

package com.library.service;
import com.library.repository.BookRepository;
public class BookService {

    private String bookName;
    private BookRepository bookRepository;

```

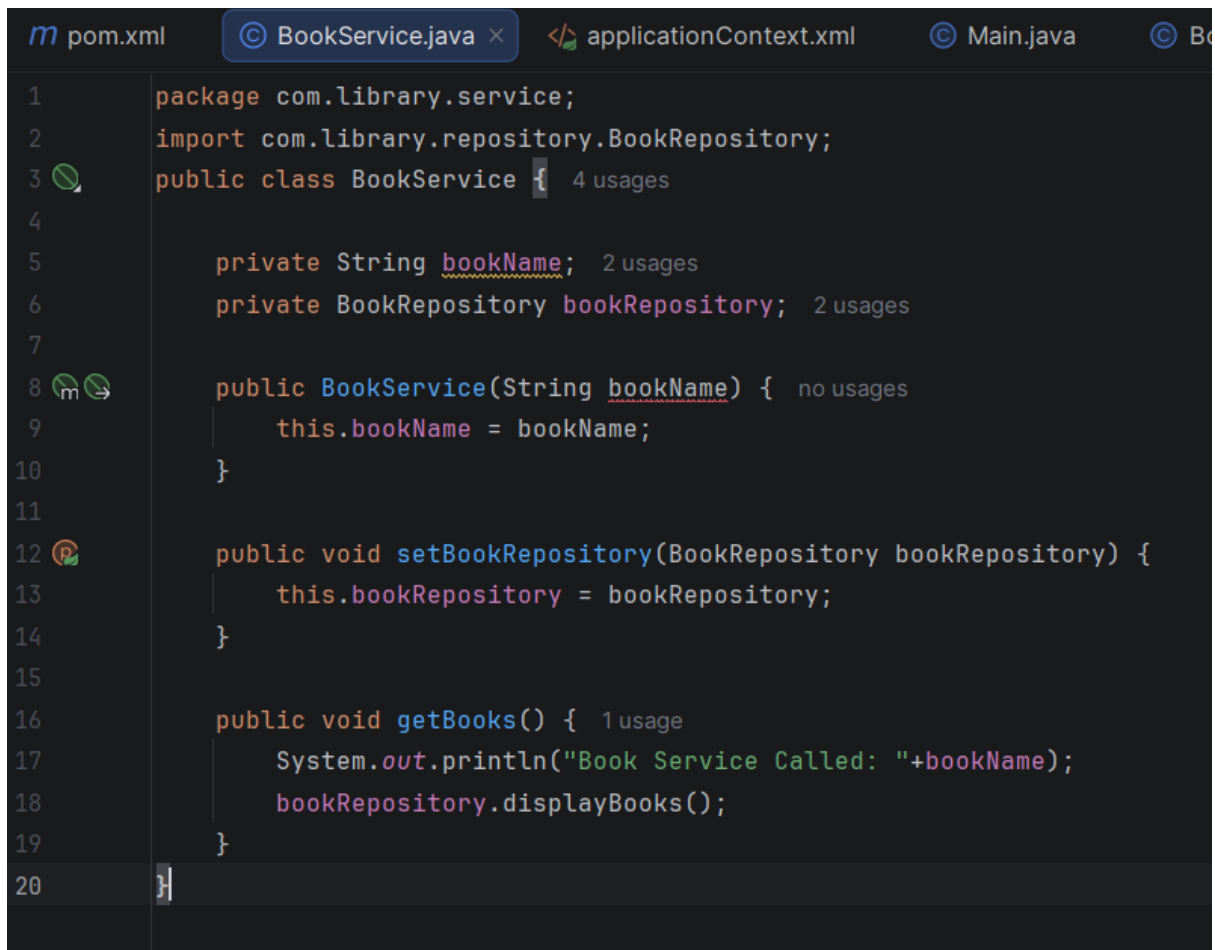
```

public BookService(String bookName) {
    this.bookName = bookName;
}

public void setBookRepository(BookRepository bookRepository) {
    this.bookRepository = bookRepository;
}

public void getBooks() {
    System.out.println("Book Service Called: "+bookName);
    bookRepository.displayBooks();
}
}

```



The screenshot shows an IDE window with the file `BookService.java` open. The code is as follows:

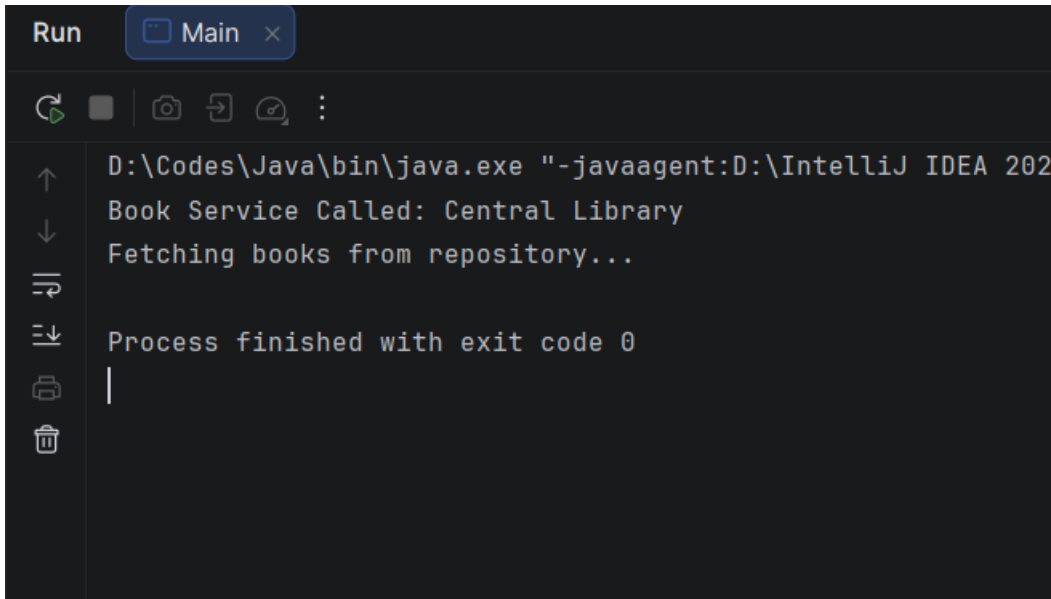
```

1 package com.library.service;
2 import com.library.repository.BookRepository;
3 public class BookService { 4 usages
4
5     private String bookName; 2 usages
6     private BookRepository bookRepository; 2 usages
7
8     public BookService(String bookName) { no usages
9         this.bookName = bookName;
10    }
11
12    public void setBookRepository(BookRepository bookRepository) {
13        this.bookRepository = bookRepository;
14    }
15
16    public void getBooks() { 1 usage
17        System.out.println("Book Service Called: "+bookName);
18        bookRepository.displayBooks();
19    }
20 }

```

The IDE interface includes tabs for `pom.xml`, `BookService.java`, `applicationContext.xml`, `Main.java`, and another `Bo` file. The `BookService.java` tab is active, showing the code with line numbers 1 through 20. The code is color-coded, and usage annotations are visible next to several lines.

OUTPUT :



```
Run Main x
D:\Codes\Java\bin\java.exe "-javaagent:D:\IntelliJ IDEA 202...
Book Service Called: Central Library
Fetching books from repository...
Process finished with exit code 0
```

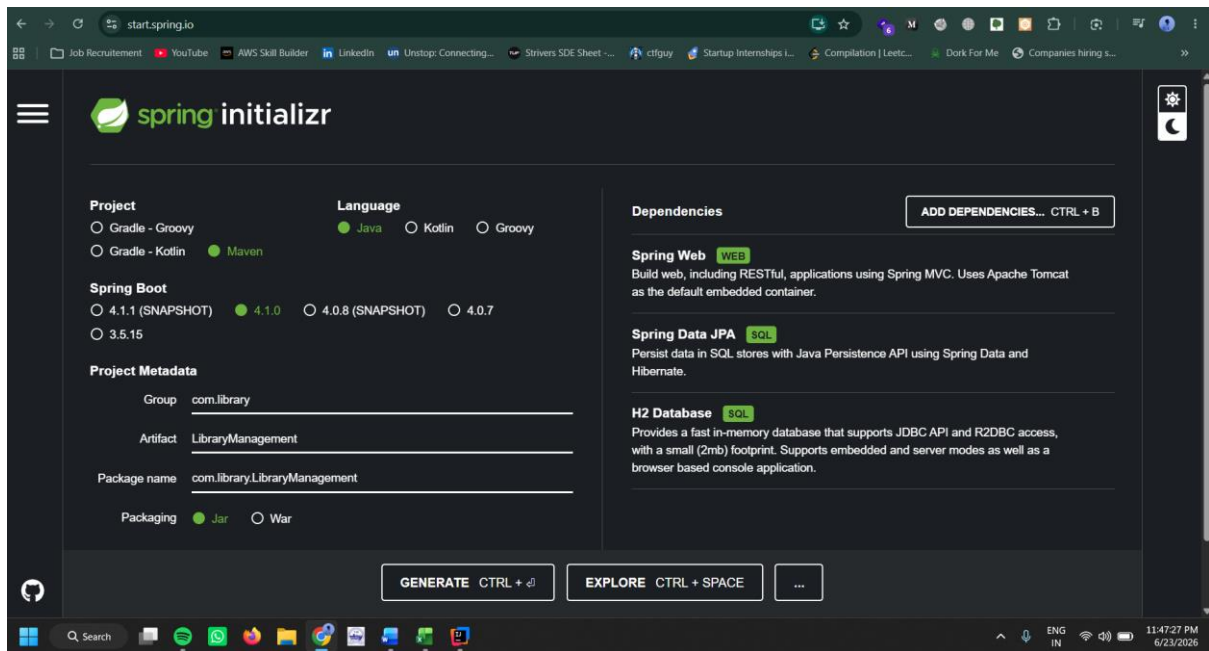
Exercise 9: Creating a Spring Boot Application

Scenario:

You need to create a Spring Boot application for the library management system to simplify configuration and deployment.

Steps:

1. **Create a Spring Boot Project:**
 - Use **Spring Initializr** to create a new Spring Boot project named **LibraryManagement**.
2. **Add Dependencies:**
 - Include dependencies for **Spring Web**, **Spring Data JPA**, and **H2 Database**.
3. **Create Application Properties:**
 - Configure database connection properties in **application.properties**.
4. **Define Entities and Repositories:**
 - Create **Book** entity and **BookRepository** interface.
5. **Create a REST Controller:**
 - Create a **BookController** class to handle CRUD operations.
6. **Run the Application:**
 - Run the Spring Boot application and test the REST endpoints.



Application.properties

spring.datasource.url=jdbc:h2:mem:librarydb

spring.datasource.driverClassName=org.h2.Driver

spring.datasource.username=sa

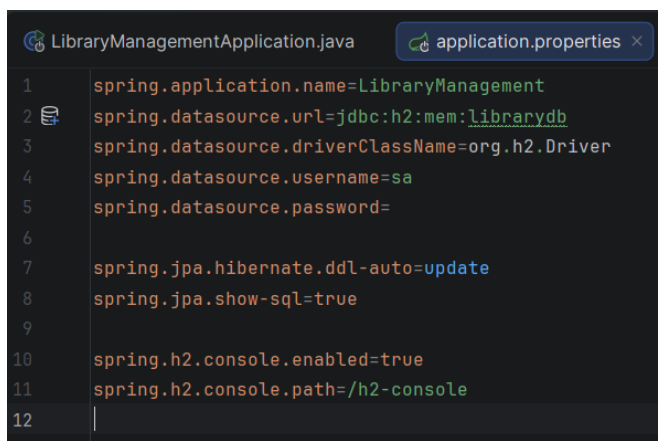
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.h2.console.enabled=true

spring.h2.console.path=/h2-console



Book.java

```
package com.library.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String author;

    public Book() {
    }

    public Book(Long id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }

    public Long getId() {
        return id;
    }

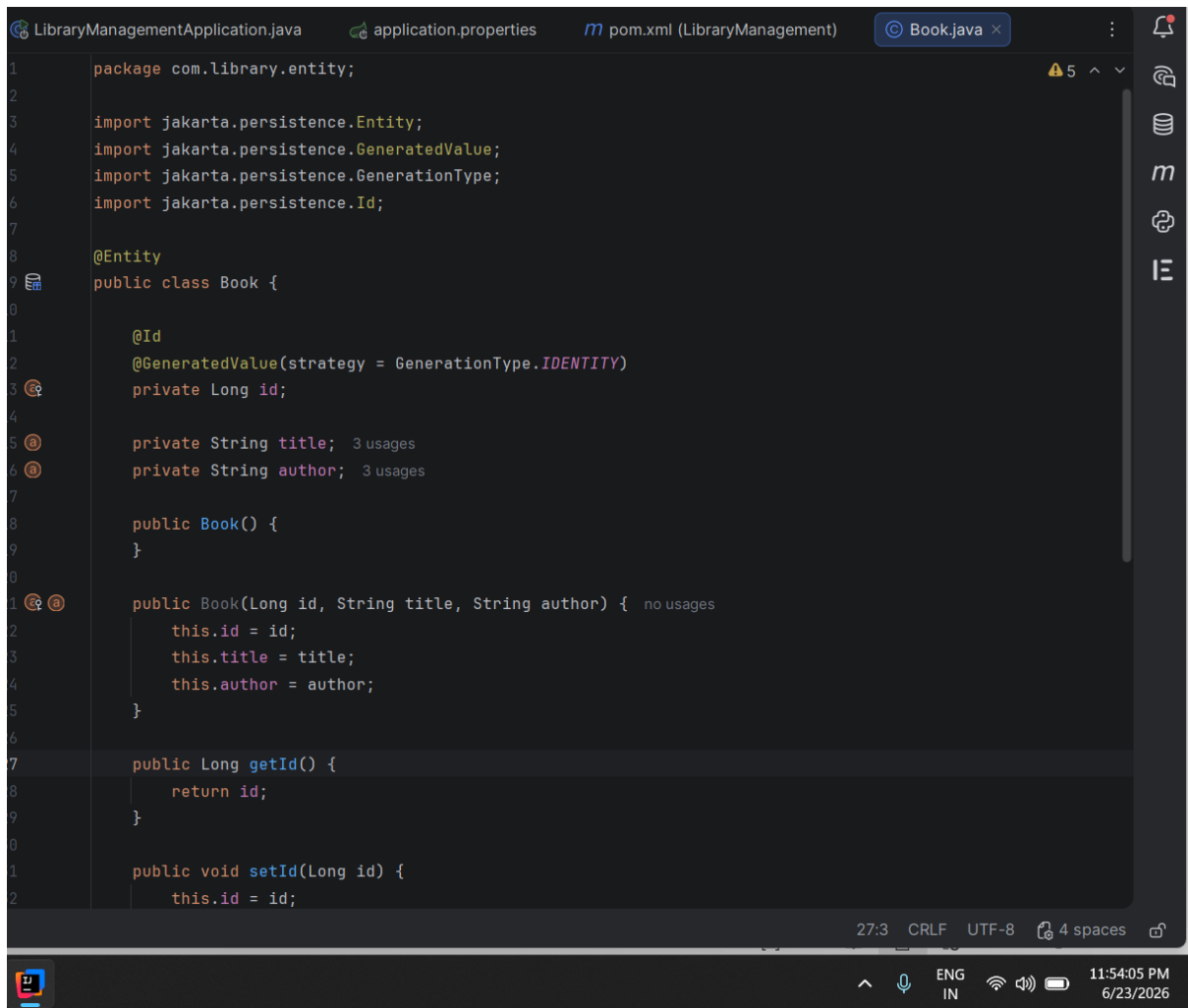
    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

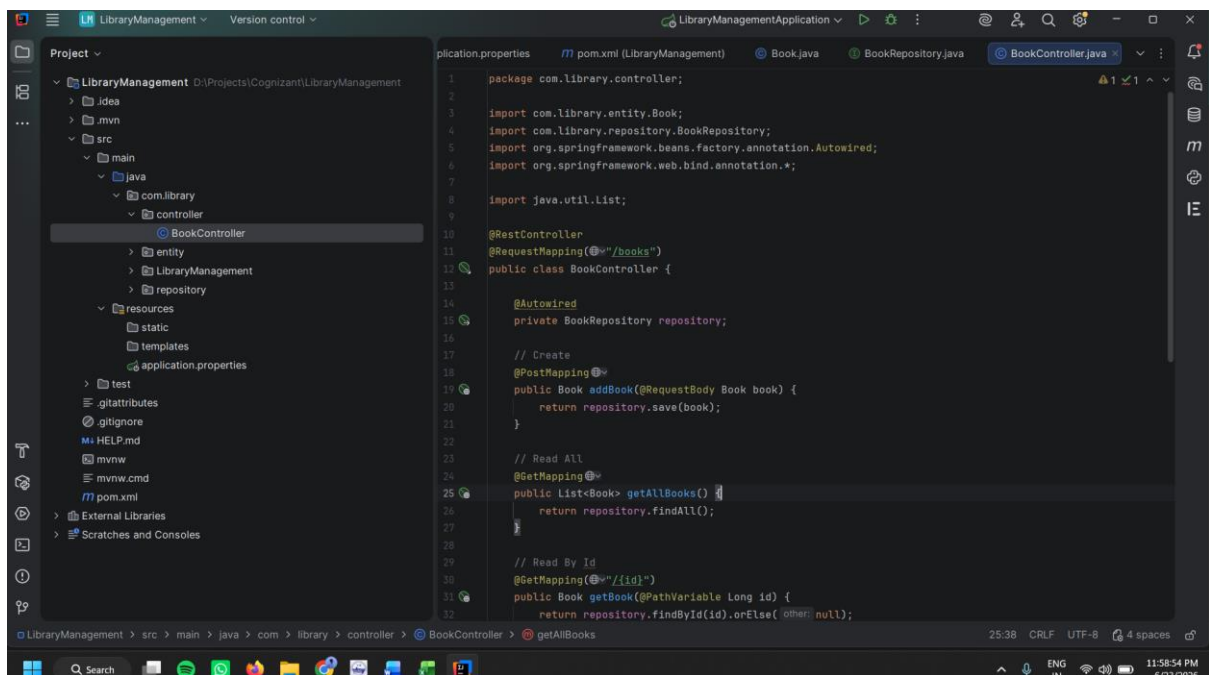
    public void setAuthor(String author) {
        this.author = author;
    }
}
```



```
1 package com.library.entity;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.GeneratedValue;
5 import jakarta.persistence.GenerationType;
6 import jakarta.persistence.Id;
7
8 @Entity
9 public class Book {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     private String title; 3 usages
16     private String author; 3 usages
17
18     public Book() {
19     }
20
21     public Book(Long id, String title, String author) { no usages
22         this.id = id;
23         this.title = title;
24         this.author = author;
25     }
26
27     public Long getId() {
28         return id;
29     }
30
31     public void setId(Long id) {
32         this.id = id;
33     }
34 }
```

27:3 CRLF UTF-8 4 spaces

11:54:05 PM 6/23/2026



```
1 package com.library.controller;
2
3 import com.library.entity.Book;
4 import com.library.repository.BookRepository;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.web.bind.annotation.*;
7
8 import java.util.List;
9
10 @RestController
11 @RequestMapping("/books")
12 public class BookController {
13
14     @Autowired
15     private BookRepository repository;
16
17     // Create
18     @PostMapping
19     public Book addBook(@RequestBody Book book) {
20         return repository.save(book);
21     }
22
23     // Read All
24     @GetMapping
25     public List<Book> getAllBooks() {
26         return repository.findAll();
27     }
28
29     // Read By Id
30     @GetMapping("/{id}")
31     public Book getBook(@PathVariable Long id) {
32         return repository.findById(id).orElse(null);
33     }
34 }
```

25:38 CRLF UTF-8 4 spaces

11:58:54 PM 6/23/2026

LibraryManagement.java:

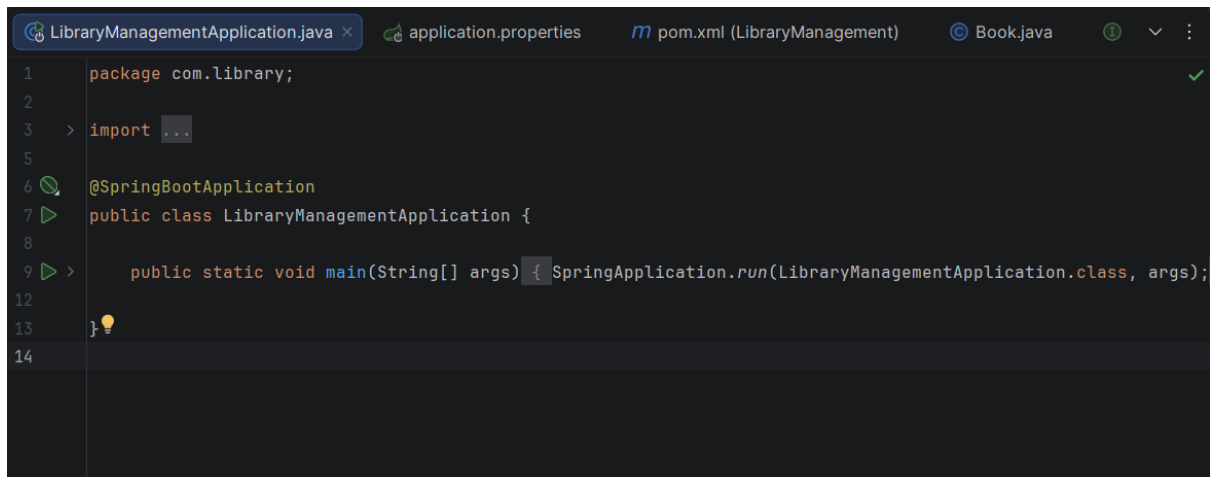
```
package com.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(LibraryManagementApplication.class, args);
    }

}
```



Output:

